🇮🇳 MyScheme Conversational AI Assistant

A **Multilingual Conversational AI Assistant** that simplifies citizen access to Indian government schemes using a **Chunked Retrieval-Augmented Generation (CRAG)** approach. This system smartly combines **Vector Search** and **LLMs (Gemini 2.0 Flash)** to deliver precise, context-rich answers in **English** and **Hindi**.

🚀 Project Overview

Navigating through thousands of Indian government schemes is complex due to scattered, verbose, and inconsistent data. This assistant bridges that gap by enabling natural conversations powered by structured retrieval and LLM reasoning.

🧠 My Approach & Thought Process

1️⃣ Understanding the Problem

Government schemes:

- Spread across **multiple ministries**.
- Contain **lengthy descriptions**, complex eligibility, and detailed application processes.
- Data is **semi-structured**, inconsistent, and often incomplete.

Why Not Direct LLM Use?

- **Context length limits** make feeding raw data impractical.
- LLMs lack **real-time structured knowledge**.

➡️ **Adopted a CRAG Pipeline**: Efficiently retrieve only what's necessary, then let the LLM generate responses.

2️⃣ Data Scraping & Cleaning — *The Foundation*

⚡ Step 1: Postman-like Scraper for Bulk Data Extraction

- Designed a scraper to:
  - Crawl **search pages**.
  - Extract **Unique IDs** for all **3300+ schemes**.
  - Automate API requests to download raw **JSON response files**.

⚡ Step 2: Handling Unstructured & Noisy Data

- Extracted essential fields from noisy, nested JSON.
- Identified missing data across schemes.

⚡ Step 3: Intelligent Data Cleaning Pipeline

- Used **Selenium + Chrome WebDriver** to scrape missing fields directly from webpages.

⚡ Step 4: Final Data Sanitization

- Marked minor missing fields as `"Not Available"`.
- Discarded entries with critical missing data.

✔ **Outcome:** A clean, structured dataset ready for preprocessing.

3️⃣ Data Preparation & Hybrid Chunking Strategy — *Optimizing for Retrieval*

⚡ The Challenge:

- Fields varied greatly in size.
- Uniform chunking caused inefficiencies.

⚡ My Hybrid Chunking Approach:

Data Type	Strategy	Why?
Small Fields (`Scheme Name` , `Description`)	**No Chunking**	Concise & self-contained.
Medium Fields (`Benefits`)	**Word-Based Chunking**	Maintain readability.
Large Fields (`Application Process`)	**Token-Aware Chunking**	Prevent token overflow.

- **Token-Aware Chunking**:
  - Split at sentence boundaries.
  - Keep within ~320 words for embedding efficiency.
- **Metadata Enrichment**:
  - `chunk_id`, `parent_doc_id`, `field` for smart retrieval.

✔ **Final Output:** ~30,000 optimized chunks.

---

4 Vector Database Design — *ChromaDB Integration*

- Used **ChromaDB** for lightweight, persistent vector storage.
- Embedded chunks using **all-MiniLM-L6-v2**.
- Batch processing for scalability.

---

5 Smart Retrieval Logic — *Precision Before Generation*

- **Top-2 Scheme Focus** to avoid context overload.
- **Action Intent Detection** for field-specific queries.
- **Sibling Retrieval** for large fields.
- Always include `Description` for grounding.

---

6 LLM Integration & Multilingual Support

- Integrated **Gemini 2.0 Flash** for fast, cost-effective responses.
- Used **Deep Translator** for seamless Hindi ↔ English handling.
- Structured prompting for clarity.

---

7 Streamlit UI — Simple Yet Effective

- Interactive chat interface.
- Session-based conversation flow.
- Expandable debug sections.

📊 Final Outcome

- ✔ **85%+ accuracy** in retrieval.
- ✔ Bilingual support.
- ✔ Optimized LLM token usage.

🛠️ Tech Stack

- **ChromaDB** – Vector Storage
- **Sentence Transformers** – `all-MiniLM-L6-v2`
- **Google Gemini 2.0 Flash** – LLM Backend
- **Streamlit** – UI
- **Deep Translator** – Language Translation
- **Selenium + Chrome WebDriver** – Web Scraping

📥 Workflow Summary

```
User Query (English/Hindi)
        |
   [If Hindi] ➔ Translate to English
        |
Vector Search (ChromaDB) using Embedded Query
        |
Smart Context Assembly (Intent + Sibling Retrieval)
        |
LLM Prompting (Gemini 2.0 Flash)
        |
   [If needed] ➔ Translate Back to Hindi
        |
Display Answer in Streamlit Chat UI
```